

Experiment No. 7

Title:

Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.

Software / Tools Required:

- Visual Studio Code (VS Code)
- Google Chrome (Browser)

Theory:

This experiment explores the Document Object Model (DOM) interface, focusing on structural traversal, dynamic element manipulation, and event handling within a web application. The core theoretical pillars implemented include:

- **Dynamic DOM Tree Manipulation:** The application utilizes methods such as `document.getElementById()` to fetch container components and `document.createElement()` to construct new elements structurally. Data insertion and node updates are executed dynamically using properties like `innerHTML`, `appendChild()`, and `replaceChild()`.
- **DOM Traversal:** To target related elements across complex interactions, the script implements structural traversal properties like `parentElement`. This allows child nodes (such as target click buttons) to cleanly identify and alter their parent list item containers (`<li>`) without relying on hardcoded unique identifiers.
- **Event Delegation:** Instead of attaching discrete event listeners to every single button added to the list, a single event listener is attached to the parent containment element (`#taskList`). The event handler uses `e.target` to determine which specific button was clicked, routing the operation appropriately.
- **State Control & Conditional Toggles:** The logic incorporates interface state alterations, managing text modifications by substituting elements with editable `<input>` text boxes. It updates conditional class listings (completed) to dynamically apply visual text decorations based on checkbox states.

AIM OF THE CASE STUDY :

To design and develop a smart, role-based to do application that helps users manage time effectively by scheduling alarms according to different activities such as study, sleep, work, fitness, and custom events, while improving productivity, routine tracking, and user convenience.

Name : Parth Vishnu

PRN : 24070521279

File Path : <file:///C:/Users/DELL/JavaScript%20lab/jscs7/parth.vishnu.index.html>

file:///C:/Users/DELL/JavaScript%20lab/jscs7/parth.vishnu.index.html



ChronoRole
Smart Role-Based Alarm Studio

Alerts: Off

+ New Alarm

Sunday, September 20, 2026

03:42:05 PM

Asia/Calcutta Timezone

NEXT ALARM IN

06h 47m 54s

Bedtime Wind-Down & Sleep

ACTIVE ROLES

STUDY

SLEEP

FITNESS

3 active alarms

Role Quick-Start Presets

Click any preset to schedule in 1 click

Study Focus Block
+25 min • Study Gong tone

Deep Revision Sprint
+50 min • Study Gong tone

Power Nap
+20 min • Gentle Chime

Night Bedtime
Today 10:30 PM • Peaceful

Work Standup / Che...
+1 hour • Digital Pulse

Posture & Hydration
+45 min • Energize Ring

All Roles 3

Study 1

Sleep 1

Work 0

Fitness 1

Custom 0

Test Ring Screen

Role Quick-Start Presets

Click any preset to schedule in 1 click

Study Focus Block
+25 min • Study Gong tone

Deep Revision Sprint
+50 min • Study Gong tone

Power Nap
+20 min • Gentle Chime

Night Bedtime
Today 10:30 PM • Peaceful

Work Standup / Che...
+1 hour • Digital Pulse

Posture & Hydration
+45 min • Energize Ring

All Roles 3

Study 1

Sleep 1

Work 0

Fitness 1

Custom 0

Test Ring Screen

Scheduled Alarms

Showing 3 of 3 alarms

STUDY

02:00 PM

in 22h 17m 29s

Deep Focus & Reading Session

Checklist: Open chapter notes, set phone on Do-Not-Disturb, and focus for 45 mins.

SMTWTFSS

SLEEP

10:30 PM

in 06h 47m 29s

Bedtime Wind-Down & Sleep

Checklist: Dim bedroom lights, put away phone, relax and prepare for deep sleep.

SMTWTFSS

FITNESS

11:00 AM

in 19h 17m 29s

Hydration & Posture Reset

Checklist: Drink a glass of water, stand up, stretch spine and shoulders.

Rings Once

Output :

This case study focuses on designing and implementing a role-based alarm application that promotes productivity, time management, and habit formation. The application will provide multiple alarm categories, live clock updates, countdowns, custom tones, and alert notifications to help users stay organized in their daily routines. By using JavaScript and web technologies, the project demonstrates how interactive systems can be created to solve real-life scheduling problems efficiently and effectively.

Result / Conclusion :

In conclusion, this experiment demonstrated real-time DOM tree manipulation and traversal by building a fully functional, empty-by-default To-Do List application. By leveraging node traversal methods (parentElement) and centralized event delegation on the parent container, the program smoothly handled adding new items, toggling visual completion states, replacing elements with inline edit boxes, and removing items on demand. The visual output confirmed the absolute correctness of the script, showcasing accurate tracking of the live task counter and immediate element updates .